TRANSPORTATION SCIENCE
Vol. 49, No. 2, May 2015, pp. 165–184
ISSN 0041-1655 (print) | ISSN 1526-5447 (online)

http://dx.doi.org/10.1287/trsc.2013.0502

Rescheduling of Railway Rolling Stock with Dynamic Passenger Flows

Leo Kroon
Rotterdam School of Management, Erasmus University Rotterdam, 3000 DR Rotterdam, The Netherlands; and Netherlands Railways, 3500 HA Utrecht, The Netherlands, lkroon@rsm.nl

Gábor Maróti
Faculty of Economics and Business Administration, VU University Amsterdam, 1081 HV Amsterdam, The Netherlands; and Netherlands Railways, 3500 HA Utrecht, The Netherlands, g.maroti@vu.nl

Lars Nielsen
Quintiq Applications, 5232 CD 's-Hertogenbosch, The Netherlands, lars.nielsen@quintiq.nl

In this paper we describe a real-time rolling stock rescheduling model for disruption management of passenger railways. Large-scale disruptions, e.g., due to malfunctioning infrastructure or rolling stock, usually result in the cancellation of train services. As a consequence, the passenger flows change, because passengers will look for alternative routes to get to their destinations. Our model takes these dynamic passenger flows into account. This is in contrast with most traditional rolling stock rescheduling models that consider the passenger flows either as static or as given input. Furthermore, we describe an iterative heuristic for solving the rolling stock rescheduling model with dynamic passenger flows. The model and the heuristic were tested on realistic problem instances of Netherlands Railways, the major operator of passenger trains in the Netherlands. The computational results show that the average delay of the passengers can be reduced significantly by taking into account the dynamic behavior of the passenger flows on the detour routes, and that the computation times of the iterative heuristic are appropriate for an application in real-time disruption management.

Keywords: passenger railways; disruption management; dynamic passenger flows; rolling stock rescheduling
History: Received: August 2011; revision received: June 2013; accepted: June 2013. Published online in *Articles in Advance* April 14, 2014.

1. Introduction

Passenger railway transportation is an important pillar of the economy in many countries. For example, more than 1.1 million passenger trips are made on an average workday in the Netherlands, a country with a population of about 16 million inhabitants. Frequent and reliable train services play an indispensable role in daily commuter transport, and provide a sustainable alternative to road mobility. Therefore several governments aim to increase the market share of railway transportation in the total mobility market.

Railway transportation starts with an extensive planning process involving line planning, timetabling, rolling stock circulation, and crew scheduling. For most parts of the planning process, model-based decision support tools have been developed; see Kroon et al. (2009). The daily railway operations are, however, often disrupted by unexpected incidents such as infrastructure or rolling stock failures or accidents. Disruption management amounts to reacting to such incidents by adjusting the timetable as well as the rolling stock and crew schedules.

Disruption management is one of the most challenging tasks in railway practice. The dispatchers have to take decisions under rapidly changing circumstances, and they need to deal with uncertainty about future events (for example, about the duration of an infrastructure or rolling stock failure). Moreover, the real-time character of the problem does not admit time for long computations or the evaluation of multiple potential solutions.

This paper focuses on large-scale disruptions. In a typical case, the railway link between two stations becomes completely or partially unavailable for a few hours because of one of the causes mentioned before. The first task in disruption management is to adjust the timetable. In the Netherlands, this task is carried out by the independent infrastructure manager ProRail, usually according to so-called *disruption scenarios*. A disruption scenario is a blueprint of the set of trains that needs to be cancelled or rerouted because of the disruption. Once the timetable has been adjusted, the train operating companies have to assign the resources rolling stock and crew to the modified timetable services: in current railway

practice, the rolling stock is rescheduled first, and then the crew schedules are adjusted accordingly.

In this paper we approach the problem from the train operating company's point of view; therefore, we assume that the adjusted timetable is given as input. Moreover, we only consider the rolling stock rescheduling problem. For an approach for solving the crew-rescheduling aspects of disruption management, we refer to Potthoff, Huisman, and Desaulniers (2010).

An important feature of disruption management is the dynamic character of passenger demand. On one hand, many passengers need to find alternative routes to their destinations, because the trains they wanted to take have been cancelled. This results in unusually large numbers of passengers in the trains on the detour routes that have not even been affected directly by the disruption. On the other hand, the actual passenger flows depend on the rolling stock circulation as well. Indeed, the seat capacity on the alternative routes may not be sufficient for the increased demand. In such a case, passengers may wait for the next train or they may be forced to choose yet another route.

We want to point out two key characteristics of our problem. First, there is no seat reservation system; within the limits set by the tariff system, the passengers are free to take any itinerary that brings them closer to their destination, and to choose the time of their journey. That is, the train operating company cannot assign the passengers to the available seat capacity. Also, if a passenger's route involves a transfer from one train to another, he cannot be sure about obtaining a seat in the second train, even if he had one in the first train.

Second, whenever a train calls at a station, only the newly boarding passengers compete for the free seats. Those passengers that were already in the train upon arrival can keep their seats until they decide to leave the train. Therefore, the passengers' boarding decisions have a temporal hierarchy: later decisions are constrained by the effects of earlier ones.

The outcome of the disruption management process is thus the result of the complex interaction between the rolling stock circulation and the passenger flows. Our goal is to design an optimization model that combines these two aspects. We measure the quality of the solution by the total passenger delay as well as by various system-related criteria, such as the number of carriage kilometers and the modifications of the shunting processes.

Several recent books and papers deal with disruption management. Yu and Qi (2004) develop a general framework for disruption management. Jespersen-Groth et al. (2010) study the practical and organizational aspects of the railway disruption management process and identify problems and solution approaches related to each step in the process. Moreover, Clausen et al. (2010) and Kohl et al. (2007) give extensive overviews of disruption management processes in the airline industry.

Most of the existing literature deals with resource-rescheduling problems under the assumption that the passenger flows are static or given. Such papers include Jespersen-Groth and Clausen (2006); Budai et al. (2009); Nielsen, Kroon, and Maróti (2012) and Cacchiani et al. (2012) for passenger railways, or Yan, Tang, and Shieh (2005) and Ball et al. (2007) for airlines.

Other papers assume that the operating company has full control of the passenger flows and can thus decide how passengers are matched with the available capacity. For example, Bratu and Barnhart (2006) study disruption management at a major airline company. Their mixed-integer programming model incorporates decisions on aircraft, crew, and passenger recovery with the options of postponing or cancelling flight legs. The objective is to simultaneously minimize operating costs, estimated passenger delay, and disruption costs.

In contrast, Dumas and Soumis (2008) propose a model for the airline booking process where the goal is to find the most profitable fleet assignment. This model is a combination of an optimization model (for the fleet assignment) and a simulation model (for the passenger flows). The simulation model, described in Dumas, Aithnard, and Soumis (2009), computes how the passengers react to the available seat capacity. In particular, they choose alternative itineraries if the most attractive flights are sold out.

Our research on railway disruption management was inspired by the approach of Dumas and Soumis (2008). The major difference is because of the quite different ways passengers can use the transportation network because of the presence or absence of a seat reservation system. Another difference is that Dumas and Soumis (2008) focus on revenue management, whereas our paper focuses on disruption management.

A particularly challenging property of real-world disruptions is the inherent uncertainty about their duration. For example, a malfunctioning signal usually needs a repair time of two to three hours. Recently, Nielsen, Kroon, and Maróti (2012) proposed a rolling horizon approach to deal with this uncertainty, which was also advantageous for the computing times.

In this paper, however, we assume that the duration of the disruption is known at the beginning of the disruption, and thereby also the adjusted timetable is assumed to be known. Note that this assumption is not really a restriction, because our method can be applied iteratively if the initial estimate of the duration of the disruption turns out to be wrong, as in

Nielsen, Kroon, and Maróti (2012). However, our goal here is to evaluate the quality of our heuristic solution approach for taking into account the dynamic passenger flows. Therefore, we want to exclude the effects of another underlying heuristic algorithm.

The contribution of this paper is summarized as follows.

- We address the disruption management problem of railway rolling stock rescheduling with special emphasis on minimizing passenger delay.
- We describe a model for simulating the passenger flows during and after a disruption.
- We combine the passenger simulation model with an optimization model for rescheduling the rolling stock.
- We provide lower bounds to evaluate the performance of the rescheduling model.
- We carry out computational experiments on realistic instances of NS.

We note that, although our model was implemented and tested on instances of NS, the approach itself is more general. The underlying rolling stock rescheduling model and all key elements of the passenger simulation model can be replaced to cover other circumstances or other assumptions on passenger behavior. Our approach is applicable to any public transport network if the passengers are not restricted by a seat reservation system.

This paper is organized as follows: In §2 we describe our model framework consisting of a simulation model and an optimization model. The assumptions on passenger behavior are described in §3. Section 4 describes the simulation of the passenger flows. Section 5 describes the feedback mechanism between the simulation model and the optimization model. Section 6 is devoted to the underlying rolling stock rescheduling model. Section 7 presents two lower bounds for assessing the quality of the obtained solutions. In §8 we present our computational results based on instances of NS. We close the paper in §9 with some conclusions and directions for further research.

2. General Framework

In this paper we investigate the interaction between rolling stock rescheduling and passenger behavior during a disruption of the railway system. The disruptions we consider are caused by a temporarily unavailable rail segment leading to the cancellation or rerouting of several trains. As a consequence, passengers have to change their routes to their destinations.

We assume that the origin-destination (OD) matrix is known and that it is the same as on a nondisrupted day. The OD matrix describes how many passengers enter the railway network at a certain location at a certain time, and their destinations. Note that detailed OD data are becoming more and more available in practice because of the introduction of smart cards in public transport.

Further, we assume that a model is available to describe how the passengers choose their travel path depending on the timetable (but not depending on the seat capacity of the trains, which is not known to the passengers). We call this choice the *travel strategy*.

This research studies the problem of adjusting the rolling stock circulation to facilitate the passenger flows. The passenger flows, however, arise from the combination of the OD matrix and the rolling stock circulation. The optimization problem can be formulated on a high abstraction level as follows:

$$\min \quad c(x) + d(y) \tag{1}$$

$$\text{subject to} \quad x \in \mathcal{X} \tag{2}$$

$$y = f(x) \in \mathcal{Y}. \tag{3}$$

Here, $\mathcal{X}$ is the set of rolling stock circulations to the modified timetable, and $\mathcal{Y}$ is the set of feasible passenger flows. The function $f\colon \mathcal{X} \to \mathcal{Y}$ gives the passenger flow that is the result when rolling stock circulation $x \in \mathcal{X}$ is operated. The objective function contains the function $c\colon \mathcal{X} \to \mathbb{R}$ for the system-related costs and the function $d\colon \mathcal{Y} \to \mathbb{R}$ for the service-related criteria.

In this paper we propose an iterative heuristic approach for solving the above model. In each iteration we carry out three steps that are sketched in Figure 1.

(i) We simulate the passenger flows with the simulation model described in §4. The simulation is based on the passengers' traveling strategies and on the capacities allocated to the trains. A traveling strategy describes how a passenger selects the route he wants to take.

(ii) We use the information on the passenger flows as feedback to the rolling stock rescheduling model; see §5. The feedback mechanism adjusts the objective function of the rescheduling model. For example, the adjusted objective function encourages additional seat capacity on trains that were too crowded in the simulated passenger flows.

(iii) We reschedule the rolling stock circulation with the rolling stock rescheduling model; see §6. Having done so, we go back to the simulation step (i) to evaluate whether the rescheduled rolling stock circulation fits better to the passenger flows.

We note that several iterations of the loop in Figure 1 may be necessary to decrease the passenger delay. This is illustrated by the following example. Suppose that the passengers' best travel path includes a transfer from a trip to another, and that both trips do

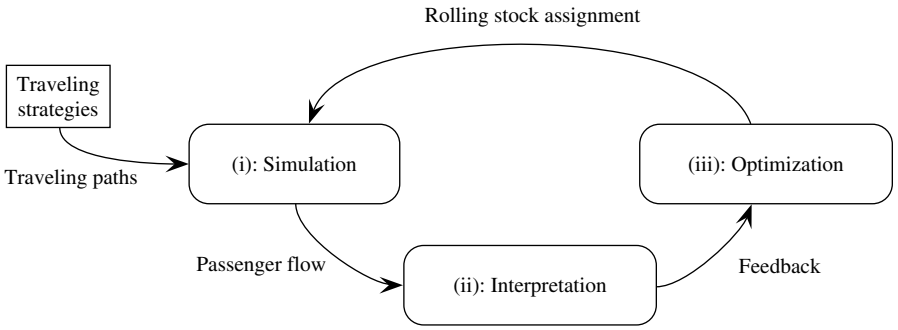

Figure 1 **Iterative Procedure for Solving the Rolling Stock Rescheduling Problem with Dynamic Passenger Flows**

not have sufficient seat capacity in the current solution. Then the feedback mechanism initially advises increasing the seat capacity of the first trip of the path only. Having followed the advice, a repeated simulation will then point out the insufficient capacity of the second trip. This seat capacity will be increased only in a next iteration of the loop in Figure 1.

In §6.4 we indicate that the first iteration can start either at the simulation step or at the optimization step. The computational results in §8 show that both methods usually lead to similar results, but starting at the simulation step requires significantly fewer iterations.

The motivation for our three-step approach is twofold. First, the model (1)–(3) is very difficult to solve in this form for real-life problem instances. One reason for this difficulty is the temporal hierarchy of the decisions in the passenger flows: later decisions are constrained by the effects of earlier ones. Another reason is that passengers act selfishly and are not controlled by the operator. Second, a fine-tuned optimization algorithm is likely to depend heavily on the particular assumptions for the passengers' behavior. We aim at designing an algorithm that is flexible enough to deal with various application-specific circumstances.

In the remainder of this paper we use the following notations. The set of stations is denoted by $\mathcal{S}$. A *trip* is a train ride without any call at the intermediate stations. The set of timetabled trips in the modified timetable is denoted by $\mathcal{T}$. A trip $t \in \mathcal{T}$ is characterized by its departure station $d(t)$, its departure time $s(t)$, its arrival station $a(t)$, and its arrival time $f(t)$. Further notations will be introduced when they are needed.

3. Modeling the Passenger Flows

In this section, we describe our model for the passenger flows. We do not consider individual passengers, but we divide them into *passenger groups*. A passenger group is a set of passengers with the same characteristics.

Passenger Groups

A passenger group p includes a given number n_p of passengers that appear in the system at a certain time instant τ_p at their origin station o_p. They all want to travel to the destination station d_p. Further, we assume that they have a certain deadline $\tilde{t}_p$: If a passenger cannot reach his destination within the deadline $\tilde{t}_p$, he will leave the system; he either gives up the journey or pursues another mode of transportation.

Throughout the whole paper we assume that passenger groups can have a fractional size; this assumption eliminates the need for upward or downward rounding rules when the passenger groups get split in the course of the passenger simulation.

The set of all passenger groups is denoted by $\mathcal{P}$. For the sake of simplicity, we assume that the time instants o_p, τ_p, and $\tilde{t}_p$ coincide with the arrival or departure time of a train.

Travel Information

The passengers plan their routes knowing the departure and arrival times of all planned timetable services. Such a planned route may contain multiple train services if there is no direct service to the destination. As was mentioned before, there is no seat reservation system.

We assume that the passengers are informed about the entire disrupted timetable as soon as the disruption starts; in particular, the passengers know which trains are cancelled and how long the disruption is going to last. Of course, the passengers do not know about the train cancellations before the disruption starts.

However, the passengers do not have information about the capacities of the trains or about the intentions of the other passengers. Thus, if they transfer from one train to another, they cannot be sure that they will get a seat in the second train.

Traveling Strategy

A passenger's traveling strategy describes how the passenger selects the route he wants to take. Note that the actual route that is followed by a passenger is

dynamic: it depends on the current time and on the passenger's current location. The traveling strategy also describes how a passenger reacts to a changing environment. Should some passenger be unable to use a train (due to a cancellation or insufficient capacity), then he remains at his current station and reconsiders his intended route, based on his knowledge of the timetable at that moment.

In this research we assume that the passengers always choose a path with the earliest possible arrival time at their destination; in the case of multiple such routes, they minimize the number of transfers from one train to another. We note, however, that our solution approach does not depend on this strategy. In fact, the framework can work with any (deterministic or probabilistic) set of route selection rules.

Interaction Between Passenger Groups

The passenger groups are assumed to be independent, selfish actors. The passenger flows emerge from the interaction between the passengers as they compete for the limited capacity of the trains.

When more passengers attempt to board a train than the available capacity allows for, only a portion of the passengers can enter the train. This is modeled by splitting passenger groups: some passengers can board the train, the rest will have to look for another travel route according to their traveling strategy.

In this research we assume that the number of passengers from each group who board a train is proportional to the size of the group. The assumption essentially states that the passenger groups are perfectly mixed on the platforms.

More formally, suppose passenger groups $p_1, \ldots, p_m$ (of size $n_{p_1}, \ldots, n_{p_m}$) attempt to board a train with a free capacity of k. Then altogether $n = \sum_{i=1}^{m} n_{p_i}$ passengers want to board the train. If $n \leq k$, then the capacity is sufficient.

However, if $n > k$, then our model allows $f_{p_i} = k \cdot n_{p_i}/n$ passengers from group p_i to board the train, whereas $n_{p_i} - f_{p_i}$ passengers are rejected by the train.

We need to point out an important assumption. When a train calls at a station, some passengers will already be in the arriving train, intending to continue in the same train. These passengers have a priority in our model; newly boarding passengers can only occupy the remaining free capacity of the train. Moreover, in our model the capacity of a train can be decreased only at the end station of the train's journey. As a consequence, once a passenger has boarded a train, he can stay in the train no matter how many calls the train is going to make until its end station. This is called the *no-forced-alight* property of the passengers' paths in the remainder of this paper.

The Passenger Graph

The journeys of the passengers are described in a graph representation of the timetable. This *passenger graph* $G = (V, A)$ is defined as follows: Starting from an empty graph, we add a node for each departure or arrival event of a trip $t \in \mathcal{T}$ at a station $s \in \mathcal{S}$. A node is thus characterized by a station $s \in \mathcal{S}$ and a time instant τ. This forms the set V of nodes.

The set of arcs consists of two kinds of arcs: *trip arcs* and *time arcs*. A trip arc joins the nodes that correspond to the trip's departure and arrival events. A time arc connects pairs of consecutive nodes at the same station:

$$A = A_{\text{trip}} \cup A_{\text{time}}$$

where

$$A_{\text{trip}} = \{((s, \tau), (s', \tau')) \in V \times V \mid \text{a train departs at time } \tau \text{ from station } s \text{ and arrives at time } \tau' \text{ at station } s'\},$$

$$A_{\text{time}} = \{((s, \tau), (s, \tau')) \in V \times V \mid \text{there does not exist a node } (s, \tilde{\tau}) \text{ with } \tau < \tilde{\tau} < \tau'\}.$$

With each arc, a capacity is associated. The capacity of a trip arc is the maximum number of passengers that can be accommodated by the assigned rolling stock. The capacity of a time arc is set to be infinite.

An individual passenger's journey corresponds to a directed path in the passenger graph. Mind, however, that members of a passenger group may have different paths; this follows from the fact that a passenger group may be split in case of scarce capacity.

The sum of the path flows over an entire passenger group p gives a network flow of value n_p. The source node of this flow is specified by the origin station o_p and by the departure time τ_p. The sink nodes are the nodes at the destination station d_p up to the deadline $\tilde{t}_p$. Nodes at time instant $\tilde{t}_p$ at other stations are also considered sinks: they allow the passengers to leave the system once their deadline expired.

The traveling strategy in our research seeks for the earliest arriving route. We use the number of transfers and the initial waiting time as tie-breaking rules. This route selection rule is easily implemented as a shortest-path query in the passenger graph. Because the passenger graph is a directed acyclic graph, such computations can be carried out very efficiently.

The Quality of the Passenger Flow

The service quality of the passenger flows can be measured by several criteria. In this study we limit ourselves to the following two aspects: (i) whether the passengers are delayed compared to their anticipated arrival time and (ii) whether the passengers arrive *at all* at their destination station within their deadline.

Passengers are delayed by the train cancellations or by the fact that some trains have insufficient capacity. Because the disrupted timetable is given, we cannot reduce the delays caused by the cancellations. Therefore, we focus on the delays caused by insufficient capacity. Thus, we measure the delays compared to the *anticipated* arrival time, which is the earliest possible arrival time in the disrupted timetable, assuming that all trains have sufficient capacity. The passengers may not be able to meet this arrival time because of insufficient capacity of the trains.

More specifically, we define the *inconvenience* of a passenger as the number of minutes of delay experienced by the passenger plus an additional penalty for exceeding the deadline. This additional penalty is set to the difference between the anticipated arrival time and the deadline. We define the *overall* service objective as the sum of the individual inconveniences. In the rest of the paper we will also simply use the term *delay* instead of inconvenience.

Example. We illustrate the assumptions by the example in Figure 2. The passenger graph contains five trips $t_1, \ldots, t_5$ from station O to D, where trips t_1, t_2, and t_4 have half an hour of travel time, and trips t_3 and t_5 take only 20 minutes. The nodes are named (s, τ) representing station s at time τ. Trip and time arcs are represented by solid and dashed arcs, respectively.

A passenger group p traveling from origin $o_p = O$ to destination $d_p = D$ starting at time $\tau_p = 10{:}00$ will thus prefer traveling with trip t_1. A passenger group p' also traveling from origin $o_{p'} = O$ to destination $d_{p'} = D$ starting at time $\tau_{p'} = 10{:}30$ will prefer waiting on time arc $((O, 10{:}30)(O, 10{:}35))$ and then travel with trip t_3 rather than with trip t_2. On the other hand, a passenger group p'' also traveling from origin $o_{p''} = O$ to destination $d_{p''} = D$ starting at time $\tau_{p''} = 11{:}00$ will travel with trip t_4 rather than wait for trip t_5, because these trips have the same arrival time, but t_4 has a shorter initial waiting time.

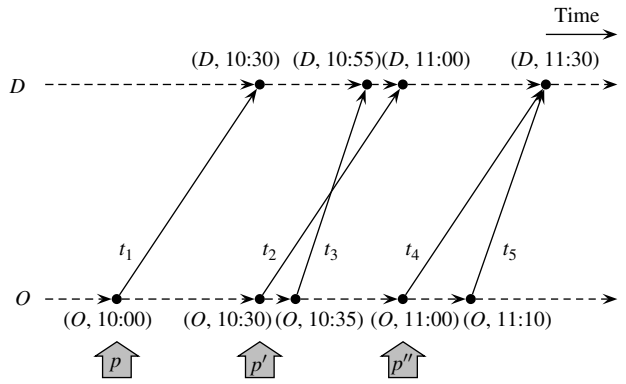

Figure 2 **Part of a Passenger Graph with Four Trips from Station *O* to Station *D***

Note. The gray arrows below the graph show when the three example passenger groups enter the system.

4. Algorithm for Simulating the Passenger Flows

In this section we explain the simulation algorithm that incorporates the assumptions on passenger behavior described in §3.

The set of passenger groups is given by $\mathcal{P}$, and we are given a passenger graph $G = (V, A)$. For the purpose of simulating the passenger flows under the above assumptions, we introduce a four-tuple (p, v, n, a). Such a four-tuple denotes n passengers from passenger group p that are positioned at node v and last traveled on trip arc a. The arc a thus indicates that the passengers are already in the train that performs arc a and do not participate in the boarding procedure if they wish to continue with the same train. If the passengers have not yet traveled with any arc, the arc a in the four-tuple is set to a dummy value $\varnothing$.

4.1. Description

The simulation algorithm acts by performing a time sweep over all trip arcs in the passenger graph and by moving the four-tuples through the graph according to the assumptions on traveling strategies and interaction. The passenger groups' travel paths correspond to a network flow in $(V; A)$. The algorithm returns a function $f\colon A_{\text{trip}} \times \mathcal{P} \to \mathbb{R}$, which is the restriction of this network flow to the set of the trip arcs. As a consequence, f is not a network flow, but can easily be extended to a network flow. The pseudocode of the algorithm is given in Figure 3.

In the preprocessing in lines 1–3, a set F of four-tuples is initialized to hold each passenger group at its origin station at the time it enters the network. The set of arcs is then ordered by departure time. If several arcs depart at the same time, then their ordering is arbitrary. The function f is set to 0 for all pairs $(a, p) \in A_{\text{trip}} \times \mathcal{P}$.

In line 4 we iterate over all arcs to apply the boarding procedure at each departure. In line 5 we denote the departure and arrival nodes of the arc a by (s, τ) and (r, σ). In line 6 we denote the preceding arc of arc a by $a_{\texttt{pred}}$. The preceding arc $a_{\texttt{pred}}$ is served by the same train immediately before arc a. This notion is necessary to determine which passengers are already in the train, because they do not have to participate in the boarding procedure. Then the subset $F' \subseteq F$ is extracted holding the four-tuples $g = (p, (s, \tau), n, b)$ representing the passengers that are situated at the departure node (s, τ) and want to travel with the involved trip. Note that this is determined by the traveling strategy of the passengers in group p. Further, $F'' \subset F'$ is the

Input: Passenger graph $G = (V, A)$, set of passenger groups $\mathcal{P}$
Output: Flow function f: $A_{\text{trip}} \times \mathcal{P} \to \mathbb{R}$
1 $F := \{(p, (o_p, \tau_p), n_p, \varnothing) \mid p \in \mathcal{P}\}$
2 $L :=$ An ordering of A_{trip} by departure time
3 Let $f(a, p) := 0$ for all $a \in A_{\text{trip}}$, $p \in P$
4 **foreach** $a \in L$ **do**
5 Let (s, τ) and (r, σ) be the departure and arrival nodes of a respectively
6 Let trip a_{pred} be the preceding trip of trip a
7 $F' := \{g = (p, (s, \tau), n, b) \in F \mid g$ wants to travel with $a\}$
8 $F'' := \{g = (p, (s, \tau), n, b) \in F' \mid g$ is already in the train (i.e., $b = a_{\text{pred}}$)$\}$
9 $k := \text{cap}(a) - \sum_{g \in F''} n(g)$
10 **foreach** $g \in F'$ **do**
11 **if** $b = a_{\text{pred}}$ **then**
12 $u := n(g)$
13 **else**
14 $u := \min\{n(g), k \cdot n(g) / \sum_{g' \in F' \setminus F''} n(g')\}$
15 $F := F \setminus \{g\}$
16 **if** $u < n$ **then**
17 Let (s, τ') be the next departure node at station s
18 $F := F \cup \{(p(g), (s, \tau'), n - u, \varnothing)\}$
19 **if** $r \neq d_p$ **then**
20 $F := F \cup \{(p(g), (r, \sigma), u, a)\}$
21 $f(a, p) := f(a, p) + u$
22 **return** f

Figure 3 Simulation Algorithm for the Passenger Flows

subset of four-tuples representing the passengers that are already in the train before it arrives.

Next, in line 9, the free capacity of arc a is calculated by subtracting from the total capacity $\text{cap}(a)$ the capacity taken up by passengers already in the train. Passengers already in the train are identified by having last traveled on arc a_{pred}.

In the loop starting from line 10, each relevant four-tuple $g \in F'$ is considered. First, the number of passengers u who obtain capacity on the train is calculated. If the passengers are already in the train (line 11), then they all fit in the train because of the *no-forced-alight* property (line 12). If not, then they participate in the boarding procedure (line 14).

The four-tuple is then removed from the set F and replaced by new four-tuples as follows. If not all passengers are assigned to arc a (i.e., $u < n$), then the $n - u$ rejected passengers from the group remain at the station at node (s, τ'), defined as the departure node of the next arc departing from this station (lines 16–18). The number of passengers u that were accepted on arc a will reappear at the arrival node (r, σ) unless it is the destination station of the passengers (lines 19–20). Finally, the flow function is updated in line 21.

Example. We return to the example in Figure 2. Let each of the trains assigned to the five trips have a capacity of 100 passengers, and let $\mathcal{P} = \{p_1, p_2\}$ consist of two passenger groups with origin $o_{p_1} = o_{p_2} = O$, destination $d_{p_1} = d_{p_2} = D$, size $n_{p_1} = n_{p_2} = 100$, and deadline sufficiently large. The groups enter the system at time $\tau_{p_1} = 10{:}30$ and $\tau_{p_2} = 10{:}35$, respectively. According to their traveling strategies, both groups prefer traveling by the path in the passenger graph that uses trip t_3. In the simulation, the four-tuples are initialized to

$$F = \{(p_1, (O, 10{:}30), 100, \varnothing), (p_2, (O, 10{:}35), 100, \varnothing)\},$$

and after processing the outgoing arcs of node $(O, 10{:}30)$ the set contains

$$F = \{(p_1, (O, 10{:}35), 100, \varnothing), (p_2, (O, 10{:}35), 100, \varnothing)\}.$$

When processing the arc of trip t_3, both groups will attempt to board the train. According to the assumptions on the boarding procedure, 50 passengers from each group will be assigned to the arc, thus setting $f(t_3, p_1) = f(t_3, p_2) = 50$. New four-tuples are inserted at the next relevant node, thus resulting in the following set of four-tuples:

$$F = \{(p_1, (O, 11{:}00), 50, \varnothing), (p_2, (O, 11{:}00), 50, \varnothing)\}.$$

When processing the arc of trip t_4, the remaining passengers will be able to board, thus setting $f(t_4, p_1) = f(t_4, p_2) = 50$.

We can calculate the delays of the involved passenger groups using the flow f. In the previous example, 50 passengers from each of the groups p_1 and p_2 suffered a delay of 35 minutes compared to their initially intended journeys, which results in a total delay of $2 \cdot 50 \cdot 35 = 3{,}500$ delay minutes. We observe that the scarcity of capacity in the trains, combined with the greedy traveling strategy of passengers, may lead to additional passenger delay. In our experiments we choose to penalize passenger delay minutes in a uniform way, although one may argue that one longer delay is worse than several smaller delays.

4.2. Implementation Issues

The traveling strategy of the passengers is implemented as a shortest-path algorithm in the passenger graph. The shortest-path search can be performed in linear time, because the passenger graph is directed and acyclic. In the implementation, we store the desired traveling path with each four-tuple to avoid recalculating the path several times. Only when passengers are rejected in the boarding procedure, their path needs to be recomputed.

The occurrence of a disruption is incorporated in the simulation algorithm by changing the passenger graph at the appropriate point in time in the simulation. All four-tuples in the set F have their preferred path recomputed then.

The simulation algorithm returns a function f: $A_{\text{trip}} \times \mathcal{P} \to \mathbb{R}$ specifying the number of passengers from

each group traveling by each trip arc. However, the feedback mechanism described in §5 needs the path decomposition of the flow. The path decomposition is easily obtained by storing the actual travel paths of each four-tuple in the course of the algorithm.

5. Feedback Mechanism

The purpose of the feedback mechanism is to interpret the passenger flows returned by the simulation algorithm, and to provide an optimization direction that is likely to improve the solution in the next iteration. This is performed by estimating the effect of the train capacities on the total passenger delay.

More formally, the feedback mechanism is a function $F\colon \mathcal{T} \times \mathbb{Z}_+ \to \mathbb{R}$ that maps the trips $\mathcal{T}$ in the modified timetable and the nonnegative integers to values in $\mathbb{R}$. For trip $t \in \mathcal{T}$ and $k \in \mathbb{Z}_+$ the value $F(t,k)$ denotes the penalty for the potential decision of assigning rolling stock with capacity k to trip t. We achieve this by considering how many passengers wanted to travel with trip t in the simulation, and by estimating the delay of any passengers who were unable to board the train at its departure.

Consider the passenger flows $f\colon A \times \mathcal{P} \to \mathbb{R}$ returned by the simulation algorithm in §4. Then consider a trip $t \in \mathcal{T}$, and let $a \in A$ be the trip arc representing trip t. From the simulation algorithm we know which passenger groups attempted to travel with arc a. Let $p_1, \ldots, p_m$ be those groups. Let $u_i = f(a, p_i)$ be the number of passengers from group p_i that traveled with arc a, and let r_i be the number of passengers from group p_i that attempted to board arc a, but were rejected.

Consider a passenger group p_i. The passengers of this group follow paths from the origin node o_{p_i} according to the path decomposition from the simulation algorithm. Suppose s_i paths contain arc a, and t_i paths result from the rejection in the boarding procedure at arc a. Then let $Q_1, \ldots, Q_{s_i}$ be the paths that contain arc a, and let $W_1, \ldots, W_{t_i}$ be the paths that result from the rejection at arc a. For a path P, let $n(P)$ be the number of passengers that traveled along path P and let $v(P)$ be the delay per passenger traveling along path P for the group p_i. Then the term

$$\texttt{TRAVEL}(p_i, a) = \frac{\sum_{j=1}^{s_i} (n(Q_j) \cdot v(Q_j))}{\sum_{j=1}^{s_i} n(Q_j)}$$

denotes the average delay per passenger from group p_i that could *travel* on arc a. Similarly, the term

$$\texttt{REJECT}(p_i, a) = \frac{\sum_{j=1}^{t_i} (n(W_j) \cdot v(W_j))}{\sum_{j=1}^{t_i} n(W_j)}$$

denotes the average delay per passenger from group p_i that were *rejected* upon boarding arc a. Using the terms $\texttt{TRAVEL}(p_i, a)$ and $\texttt{REJECT}(p_i, a)$, we estimate the contribution per passenger to the total delay by

$$\texttt{AVG\_COST}(a) = \frac{\sum_{i=1}^{m} r_i \cdot (\texttt{REJECT}(p_i, a) - \texttt{TRAVEL}(p_i, a))}{\sum_{i=1}^{m} r_i}.$$

Thus, $\texttt{AVG\_COST}(a)$ can be interpreted as the marginal increase of the total passenger delay when the capacity of arc a is decreased from its current value by one unit. We set the function $F(t,k)$ in the feedback mechanism to

$$F(t,k) = \max\left\{0, \sum_{i=1}^{m} (u_i + r_i) - k\right\} \cdot \texttt{AVG\_COST}(a),$$

where a is the trip arc corresponding to trip t. The value $F(t,k)$ estimates the increase of the passenger delay (if any) in case the capacity of trip t is changed to k from its current value.

Example. Consider the example passenger flow of a passenger group p in the left diagram of Figure 4. The group consists of 100 passengers traveling from origin O to destination D, starting at 9:00, and having deadline 13:15. Upon boarding arc $a = (A, 10{:}10)(B, 10{:}30)$ the passenger group is split, where 70 passengers are able to board and the remaining 30 passengers are rejected. The group is further split at some other arcs resulting in the paths shown in the right panel of Figure 4. Paths Q_1 and Q_2 follow from the boarding of arc a, whereas paths W_1 and W_2 follow from the rejection at arc a.

Assume the passengers in group p had expected arrival time of 11:45. Then we derive the following path sizes and values from the figure: $n(Q_1) = 50$, $v(Q_1) = 0$, $n(Q_2) = 20$, $v(Q_2) = 60$, $n(W_1) = 10$, $v(W_1) = 60$, $n(W_2) = 20$, and $v(W_2) = 90$. The average contribution per passenger to the total delay for passengers from group p, respectively, traveling with or rejected at arc a, is calculated as follows:

$$\texttt{TRAVEL}(p, a) = \frac{\sum_{j=1}^{2} (n(Q_j) \cdot v(Q_j))}{\sum_{j=1}^{2} n(Q_j)} = \frac{50 \cdot 0 + 20 \cdot 60}{70} = 17.1,$$

$$\texttt{REJECT}(p, a) = \frac{\sum_{j=1}^{2} (n(W_j) \cdot v(W_j))}{\sum_{j=1}^{2} n(W_j)} = \frac{10 \cdot 60 + 20 \cdot 90}{30} = 80.$$

Thus, the estimated contribution to the delay of the passenger flow per passenger rejected at arc a is $80 - 17.1 = 62.9$.

6. Rolling Stock Rescheduling

In this section we give a brief description of the model that we use for rescheduling the rolling stock. Our

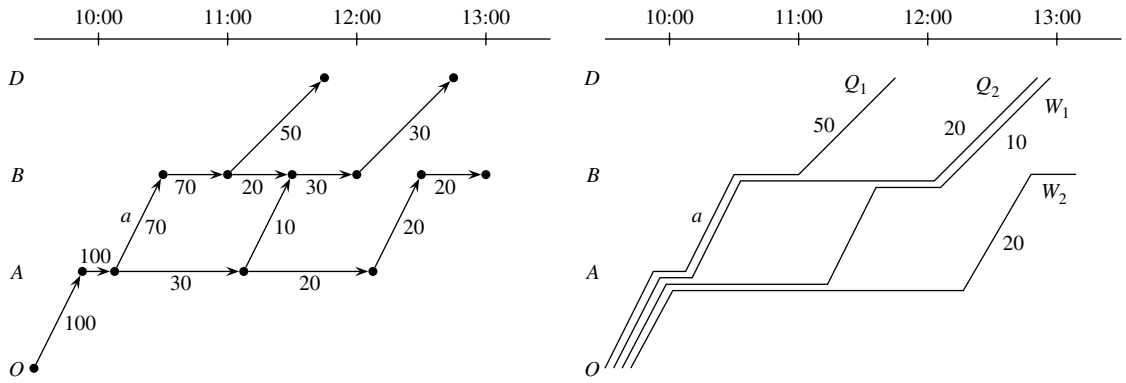

Figure 4 **Left: The Passenger Flow for Passenger Group p_i; Right: The Path Decomposition**

approach is based on the same ideas as those found in Nielsen, Kroon, and Maróti (2012) and in Cacchiani et al. (2012). All of these three rescheduling models are in turn extensions of the model described by Fioole et al. (2006).

6.1. Definitions

Recall that the set of stations is denoted by $\mathcal{S}$ and the set of trips by $\mathcal{T}$. The latter set contains the trips of the timetable after it has been modified according to the relevant disruption scenario. Furthermore, we consider *aggregated* trips that depart from and arrive at a station where the train compositions can be modified. The time horizon covered by the model is typically one day. Thus, the set $\mathcal{T}$ contains all trips (of a certain set of lines) that are carried out on a single day. Each trip $t \in \mathcal{T}$ has a successor trip $\nu(t)$, which describes the next trip that will be covered by the (majority of the) rolling stock that covered trip t. Between trips t and $\nu(t)$, the composition of the train may be modified by a shunting operation.

In the rolling stock rescheduling model, the set of *train unit types* is denoted by M. A train *composition* is a sequence of train units that have been coupled to each other to compose a single train. The set of all possible train compositions is denoted by C. For example, if a and b are train unit types, then aab and aba are different compositions of three train units with the same capacity. We also introduce the set K as the set of composition changes $(c \rightarrow c')$ from composition c to composition c' involving a shunting operation. That is, at least one train unit is coupled to or uncoupled from composition c to get composition c'.

The originally planned rolling stock circulation is described in terms of the parameters c_t^0. That is, according to the original rolling stock circulation, composition $c_t^0 \in C$ is assigned to trip t. The planned start-of-day and end-of-day inventories of rolling stock type m at station s are denoted by $i_{s,m}^0$ and $i_{s,m}^\infty$, respectively. We assume that the disruption starts at time instant τ and has a duration δ. Thus, for all trips that depart before time instant τ, the actually allocated composition is the same as the planned composition.

The main binary decision variables are the variables $X_{t,c}$, expressing whether composition c is assigned to trip t. Moreover, we have binary variables $Z_{t,c,c'}$ whose value is 1 if composition c is assigned to trip t and composition c' is assigned to the successor trip $\nu(t)$ of trip t. These Z variables are only defined for those triples (t, c, c') where the composition change from composition c to c' is allowed between trips t and $\nu(t)$. Thus, the local constraints on the composition changes are represented by these Z variables.

The stations are modeled at an aggregate level as *inventories* of train units. The inventory of a station at a certain time instant consists of all train units that are temporarily located at that station, because they have not been assigned to any trip then. Train units coupled to a train are subtracted from the inventory, and train units uncoupled from a train are added to the inventory. The end-of-day inventories of station s of type m are represented by the variables $I_{s,m}^\infty$. Deviations from the intended end-of-day inventories $i_{s,m}^\infty$ are denoted by the variables $D_{s,m}^+$ and $D_{s,m}^-$.

The parameters $\alpha_{t,c,c',s,m}$ and $\beta_{t,t',c,c',m}$ indicate how the inventory at a station increases or decreases as a result of the shunting operations: If the shunting operation at station $a(t)$ between trips t and $\nu(t)$ that transforms composition c into c' is an uncoupling operation of one train unit of type m, then $\alpha_{t,c,c',a(t),m} = 1$; if it is a coupling operation, then it is -1; if $s \neq a(t)$, then $\alpha_{t,c,c',s,m} = 0$. The definition of the parameter $\beta_{t,t',c,c',m}$ is similar to a large extent. Therefore, we omit the details of their definition.

6.2. Model Description

Given these definitions, the model that is used for rescheduling the rolling stock in the ith iteration of

the algorithm shown in Figure 1 can be described as follows:

$$\min \quad c(X,Z,D)+g_i(X) \tag{4}$$

$$\text{s.t.} \quad \sum_{c\in C} X_{t,c}=1 \quad \forall t\in\mathcal{T}, \tag{5}$$

$$X_{t,c_t^0}=1 \quad \forall t\in\mathcal{T}\colon d(t)<\tau, \tag{6}$$

$$X_{t,c}=\sum_{c'\in C} Z_{t,c,c'} \quad \forall t\in\mathcal{T}, c\in C, \tag{7}$$

$$X_{\nu(t),c'}=\sum_{c\in C} Z_{t,c,c'} \quad \forall t\in\mathcal{T}, c'\in C, \tag{8}$$

$$I^{\infty}_{s,m}=i^0_{s,m}+\sum_{t\in T}\sum_{(c\to c')\in K}\alpha_{t,c,c',s,m}\times Z_{t,c,c'} \quad \forall s\in\mathcal{S}, m\in M, \tag{9}$$

$$i^0_{d(t),m}+\sum_{t'\in T}\sum_{(c\to c')\in K}\beta_{t,t',c,c',m}\times Z_{t',c,c'}\geq 0 \quad \forall t\in\mathcal{T}, m\in M, \tag{10}$$

$$I^{\infty}_{s,m}=i^{\infty}_{s,m}+D^{+}_{s,m}-D^{-}_{s,m} \quad \forall s\in\mathcal{S}, m\in M, \tag{11}$$

$$X_{t,c},\ Z_{t,c,c'} \text{ binary} \quad \forall t\in\mathcal{T}, c\in C, c'\in C, \tag{12}$$

$$I_{t,m},\ y^{\infty}_{s,m}\geq 0, \text{ integer} \quad \forall t\in\mathcal{T}, s\in\mathcal{S}, m\in M. \tag{13}$$

The objective function (4) is explained in detail in §6.3. Constraints (5) state that each trip must be assigned exactly one composition. Constraints (6) guarantee that no composition is modified before the disruption starts. Constraints (7) state that, if composition c is assigned to trip t, then the composition c' on the successor trip $\nu(t)$ is selected from one of the compositions that can be reached from composition c via an allowed shunting operation. Similarly, constraints (8) link the composition on a trip to a possible composition on the predecessor trip.

Constraints (9) describe the final inventory $I^{\infty}_{s,m}$ of train units of type m at station s in terms of the initial inventory $i^0_{s,m}$ plus the increases and decreases of the inventory depending on the local shunting operations. In a similar way, constraints (10) express the increases and decreases of the inventory of train units of type m at the stations during the whole day, imposing that this inventory should always be nonnegative. Finally, constraints (11) describe the deviations of the rolling stock inventories at the stations by the end of the day. The remaining constraints describe the domains of the decision variables.

6.3. Objective Function

The linear objective function (4) of the rolling stock rescheduling model consists of two parts: the system-related costs $c(X,Z,D)$ and the service-related costs $g_i(X)$. The system-related costs include penalties for modifications in the rolling stock compositions, modifications in the shunting operations, and end-of-day off-balances (represented by the variables X, Z, and D, respectively).

One could define the service-related costs in the ith iteration by the function $g_i(X)$ using the feedback mechanism $F\colon \mathcal{T}\times\mathbb{Z}_+\to\mathbb{R}$; that is,

$$g_i(X)=\sum_{t\in\mathcal{T}}\sum_{c\in C} F(t,\mathtt{cap}(c))\cdot X_{t,c}.$$

However, we experienced that this approach could lead to cyclical behavior, because the feedback from earlier iterations is ignored in this way. We therefore modified the part of the objective function derived from the feedback into the following:

$$g_i(X)=(1-\alpha)g_{i-1}(X)+\alpha\sum_{t\in\mathcal{T}}\sum_{c\in C} F(t,\mathtt{cap}(c))\cdot X_{t,c}, \tag{14}$$

where $0\leq\alpha\leq 1$ is a parameter that weighs the last feedback against feedback from earlier iterations. This way, the feedback from a certain iteration gradually becomes less relevant. Note that $\alpha=1$ describes the special case where feedback from earlier iterations is ignored completely. The similar introduction of such a parameter α in the approach by Dumas and Soumis (2008) was crucial for the performance of their solution procedure. Note that (14) is similar to an exponential smoothing model that is used for forecasting time series.

6.4. The First Iteration

The iterative algorithm described in §2 consists of a main iteration loop. There are several possible choices for entering the loop. We experimented with two variants of the algorithm.

In the Boot-Sim variant, as Figure 1 suggests, we start the iterative algorithm's main loop with a simulation step. When running the simulation for the first time, we assume that each trip is assigned the highest possible number of seats on that trip. The outcome of the first simulation run gives an indication of the per-trip seat demand. Note that the feedback mechanism described in §5 is meaningless after the first simulation run, because no feasible rolling stock assignment is available yet. Therefore, the first iteration differs from later iterations.

After the first simulation run, we extend the system costs $c(X,Z,D)$ by adding penalty terms for seat shortages with respect to the anticipated demand, and we compute the rolling stock assignment subject to the system costs. From this point on, we follow the simulation, feedback, and optimization steps as described in §2. Mind that the additional seat shortage penalty terms are kept in the system costs during all iterations.

We also propose the alternative approach Boot-Opt for the first iteration: we enter the main iteration loop at the rolling stock optimization step, thereby minimizing the system costs without considering the modifications of the seat demand.

7. Lower Bounds

To assess the quality of our solutions, we describe two methods for constructing lower bounds for the solution value. This allows us to analyze how much of the delays is caused by the modifications in the disrupted timetable and how much is caused by the shortage of capacity.

7.1. Lower Bound Based on Unlimited Capacity

An intuitive way to relax the underlying assumptions of the system on passenger interaction is to assume that all trains have unlimited capacity. This way, no passenger is ever rejected in the boarding procedure, and all passengers can follow the best path in the passenger graph according to their traveling strategy, given the actual modified timetable. This implies that all delays experienced by the passengers are caused by modifications to the timetable (cancelled trains) rather than by insufficient capacities of the trains.

This relaxation corresponds to the situation where unlimited amounts of rolling stock are available, an arbitrary amount of rolling stock may be assigned to each train, and there are no limitations on the shunting possibilities at any station. It results in a lower bound on the service-related cost, i.e., the total amount of delay experienced by the passengers in the system. We call this lower bound the *infinite capacity lower bound* (IC-LB).

Calculating the IC-LB simply amounts to simulating the passenger groups $\mathcal{P}$ in a passenger graph $G = (V, A)$ where all arcs $a \in A$ have capacity $\mathtt{cap}(a) = \infty$.

7.2. Lower Bound Based on Centralized Passenger Flows

Trips right after the beginning of the disruption are potential bottlenecks: they are likely to see an elevated passenger demand, but they are unlikely to be able to receive additional seat capacity. In this section we propose a lower bound that can deal with the limited capacity of these potential bottlenecks trips.

The passengers themselves choose their routes through the railway network in a greedy way. Therefore, the resulting passenger flows may be suboptimal with respect to the total delay of the passengers. A lower bound on the passenger delay is obtained by assuming that the train operating company optimally allocates the passengers to the trains.

For passenger group $p \in \mathcal{P}$ we denote by π_p the set of paths on which the passengers in the group can travel. The paths originate from the node (o_p, τ_p) in the passenger graph $G = (V, A)$ and can lead to any station in the network—not only to the destination station.

We define for each passenger group $p \in \mathcal{P}$ and possible traveling path $q \in \pi_p$ a continuous variable $Y_{p,q} \in \mathbb{R}_+$. This variable represents the number of passengers from group p that travel with path q. The fact that the variable $Y_{p,q}$ is continuous is in line with our earlier assumption that the passenger groups can have a fractional size.

Let $c_{p,q}$ denote the delay by one passenger from group p traveling with path q. Furthermore, as before, the binary variables $X_{t,c} \in \{0, 1\}$ model the assignment of train composition c to trip t as in the rolling stock rescheduling model described in §6. Then the passenger flow with operator control may be expressed as the following integer linear program:

$$\min \sum_{p \in \mathcal{P}} \sum_{\substack{q \in \pi_p: \\ a \in q}} c_{p,q} Y_{p,q} \tag{15}$$

$$\text{subject to} \sum_{q \in \pi_p} Y_{p,q} = n_p \quad \forall p \in \mathcal{P}, \tag{16}$$

$$\sum_{p \in \mathcal{P}} \sum_{\substack{q \in \pi_p: \\ a \in q}} Y_{p,q} \le \sum_{c \in C} \mathtt{cap}(c) X_{t,c} \quad \forall a \in A, \quad t \text{ is trip of } a, \tag{17}$$

$$X \in \mathcal{X}, \tag{18}$$

$$Y_{p,q} \in \mathbb{R}_+ \quad \forall p \in \mathcal{P}, q \in \pi_p. \tag{19}$$

The objective function (15) expresses that the aim is to minimize the total passenger delay. Constraints (16) state that all passengers in a group must travel by one of the possible paths. The capacity constraints (17) denote that the number of passengers traveling on an arc is limited by the capacity that is determined by the rolling stock composition on the corresponding trip. Constraints (18) state that the vector X of composition variables must belong to the set $\mathcal{X}$ of feasible rolling stock circulations. We call this the *strong operator controlled lower bound* (SOC-LB). However, we did not solve this model, because it turned out to be too complex.

Here we propose a somewhat weaker lower bound by relaxing constraints (17)–(18). The relaxation amounts to replacing the right-hand side value of (17) by the largest-possible capacity on the trip in question. That is, we replace constraints (17)–(18) by

$$\sum_{p \in \mathcal{P}} \sum_{\substack{q \in \pi_p: \\ a \in q}} Y_{p,q} \le \max\{\mathtt{cap}(c) \mid c \in C, x \in \mathcal{X}, X_{t,c} = 1\} \quad \forall a \in A, \quad t \text{ is trip of } a. \tag{20}$$

The right-hand side values of (20) are easily derived from the requirements of the underlying rolling stock scheduling problem. The proposed relaxation reduces the complexity of the model (15)–(19) by turning it into a fractional multicommodity flow problem, but, obviously, it also weakens the lower bound. We call the resulting solution the *weak operator controlled lower bound* (WOC-LB). The multicommodity flow problem is solved using a textbook column generation procedure (Ahuja, Magnanti, and Orlin 1993).

It is not difficult to see that the WOC-LB is at least as strong as IC-LB. Indeed, if one relaxes the train capacity constraints (20) of WOC-LB, then the operator control assigns each passenger to the best path in the passenger graph according to his traveling strategy. This also happens in IC-LB. Therefore, the infinite capacity relaxation of WOC-LB coincides with IC-LB.

8. Computational Experiments

In this section we describe the computational experiments that were performed based on our approach. In §8.1 we describe how we generated the test instances, and in §8.2 we report and discuss the results.

The computational experiments were performed on a desktop computer with an Intel Core i7-2600 3.40 GHz CPU and with 16 GB of RAM, running the 64-bit edition of Windows 7. All algorithms were implemented in Java, using the optimization software CPLEX release 12.4.

8.1. Instances

For testing our approach, we constructed a number of instances from realistic data based on the Intercity network of NS in 2009. The instances involve the heavily utilized core part of the Dutch railway network connecting the 14 stations shown in Figure 5. This part of the network is serviced by the 16 Intercity lines that are listed in the figure as well. The lines call at the given stations and operate with the specified frequencies. On most routes there are at least four Intercity trains per hour between each pair of neighboring stations.

We note that some of the involved lines in reality continue beyond the terminal stations shown in Figure 5, i.e., to the southwest of Dordrecht (Ddr), north of Alkmaar (Amr) and Zwolle (Zl), east of Deventer (Dv), southeast of 's-Hertogenbosch (Ht), and south of Arnhem (Ah).

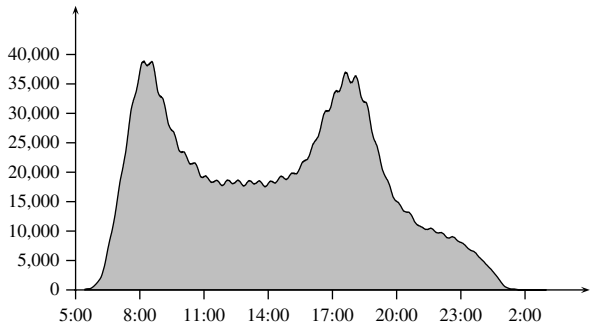

Figure 6 **Number of Passengers in the System During the Day in the Undisrupted Situation**

The initial timetable is from a workday and contains 2,324 trips. All trips are assumed to be served with rolling stock of the type VIRM. These are train units that are available in two variants with four and six carriages, respectively. The two variants have maximum technical capacities of 572 and 847 passengers per unit, respectively. The maximum train length is assumed to be 14 carriages on all trips, and it is possible for all lines to perform shunting operations at the terminal stations except Utrecht (Ut).

Figure 6 shows the number of passengers in the system during the day in the instances. We observe that the peak hours around 8:00 and 17:00 are the busiest hours of the day, and therefore these are the most likely periods to experience capacity problems.

We created the passenger groups for the instances by matching passenger counts on each trip with OD data, which resulted in 11,415 passenger groups for the full-day instance. To construct the deadlines of the passenger groups, we assume that passengers are willing to accept at most an increase in travel time of 50% plus 90 minutes.

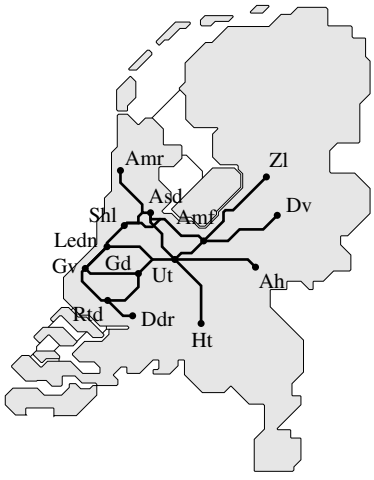

Figure 5 **The Network Considered in the Test Instances**

Line	Stations	Frequency
500	Gv Gd Ut Amf Zl	Hourly
700	Shl Amf Zl	Hourly
800	Amr Asd Ut Ht	Half hourly
1500	Asd Amf Dv	Half hourly
1600	Shl Amf Dv	Hourly
1700	Gv Gd Ut Amf Dv	Hourly
1900	Gv Rtd Ddr	Half hourly
2000	Gv Gd Ut Ah	Half hourly
2100	Asd Shl Ledn Gv Rtd Ddr	Half hourly
2600	Asd Shl Ledn Gv	Half hourly
2800	Rtd Gd Ut Amf	Half hourly
3000	Amr Asd Ut Ah	Half hourly
3500	Shl Ut Ht	Half hourly
8800	Ledn Ut	Half hourly
20500	Rtd Gd Ut	Hourly
21700	Rtd Gd Ut	Hourly

Table 1 Instances for the Computational Experiments

Instance	Disruption	Time	Assigned units	Reserve units	Passengers	Passenger groups
D1T1O1R1P1	Rtd-Gv	16:00–19:00	114, 44	0	422,022	11,415
D1T1O1R2P1	Rtd-Gv	16:00–19:00	114, 44	1, 2	422,022	11,415
D1T1O1R3P1	Rtd-Gv	16:00–19:00	114, 44	4, 2	422,022	11,415
D1T1O2R1P1	Rtd-Gv	16:00–19:00	106, 41	0	422,022	11,415
D1T1O1R2P2	Rtd-Gv	16:00–19:00	114, 44	1, 2	485,375	11,415
D1T2O1R1P1	Rtd-Gv	11:00–15:00	114, 44	0	422,022	11,415
D2T1O1R1P1	Gd-Ut	16:00–19:00	114, 44	0	422,022	11,415
D2T1O1R2P1	Gd-Ut	16:00–19:00	114, 44	1, 2	422,022	11,415
D2T1O1R3P1	Gd-Ut	16:00–19:00	114, 44	4, 2	422,022	11,415
D2T1O2R1P1	Gd-Ut	16:00–19:00	106, 41	0	422,022	11,415
D2T1O1R2P2	Gd-Ut	16:00–19:00	114, 44	1, 2	485,375	11,415
D2T2O1R1P1	Gd-Ut	11:00–15:00	114, 44	0	422,022	11,415
D3T1O1R1P1	Ut-Amf	16:00–19:00	114, 44	0	422,022	11,415
D3T1O1R2P1	Ut-Amf	16:00–19:00	114, 44	1, 2	422,022	11,415
D3T1O1R3P1	Ut-Amf	16:00–19:00	114, 44	4, 2	422,022	11,415
D3T1O2R1P1	Ut-Amf	16:00–19:00	106, 41	0	422,022	11,415
D3T1O1R2P2	Ut-Amf	16:00–19:00	114, 44	1, 2	485,375	11,415
D3T2O1R1P1	Ut-Amf	11:00–15:00	114, 44	0	422,022	11,415
D4T1O1R1P1	Gv-Ledn	16:00–19:00	114, 44	0	422,022	11,415
D4T1O1R2P1	Gv-Ledn	16:00–19:00	114, 44	1, 2	422,022	11,415
D4T1O1R3P1	Gv-Ledn	16:00–19:00	114, 44	4, 2	422,022	11,415
D4T1O2R1P1	Gv-Ledn	16:00–19:00	106, 41	0	422,022	11,415
D4T1O1R2P2	Gv-Ledn	16:00–19:00	114, 44	1, 2	485,375	11,415
D4T2O1R1P1	Gv-Ledn	11:00–15:00	114, 44	0	422,022	11,415
D5T1O1R1P1	Asd-Ut	16:00–19:00	114, 44	0	422,022	11,415
D5T1O1R2P1	Asd-Ut	16:00–19:00	114, 44	1, 2	422,022	11,415
D5T1O1R3P1	Asd-Ut	16:00–19:00	114, 44	4, 2	422,022	11,415
D5T1O2R1P1	Asd-Ut	16:00–19:00	106, 41	0	422,022	11,415
D5T1O1R2P2	Asd-Ut	16:00–19:00	114, 44	1, 2	485,375	11,415
D5T2O1R1P1	Asd-Ut	11:00–15:00	114, 44	0	422,022	11,415

8.1.1. Disruptions. The disruptions considered in the computational experiments all concern situations where a certain part of the network is unavailable for several hours. The timetable is modified according to current practice by cancelling affected trips and by turning trains on either side of the disruption.

The instances are listed in Table 1. Each instance is named by a string like "D2T1O1R2P1" characterizing the instance. The name consists of five parts, where the first part of the name "D*i*" describes the place of the disruption; "D1" is between Rotterdam (Rtd) and The Hague (Gv), "D2" is between Gouda (Gd) and Utrecht (Ut), "D3" is between Utrecht (Ut) and Amersfoort (Amf), "D4" is between The Hague (Gv) and Leiden (Ledn), and "D5" is between Amsterdam (Asd) and Utrecht (Ut).

The second part of the name "T*i*" describes the time interval of the disruption; for instances with "T1" the blockage lasts from 16:00 to 19:00 in the afternoon peak period, and for instances with "T2" the blockage lasts from 11:00 to 15:00 in the offpeak period.

The third part of the name "O*i*" describes the original rolling stock circulation used for the undisrupted situation. Instances with "O1" are based on a circulation with 114 rolling stock units of the variant with four carriages and 44 rolling stock units of the variant with six carriages. The original circulation is planned such that enough capacity is assigned to all trips to accommodate all passengers in the undisrupted situation. Furthermore, every train has significant slack capacity compared to the "full" capacity. In the instances with "O2" only 106 and 41 units of the two respective train unit types are available.

The fourth part of the name "R*i*" refers to the number of available reserve units. Instances with "R1" do not have reserve units. Instances with "R2" and "R3" have three (1 and 2 of the two types), and six (4 and 2 of the two types) reserve units, respectively. The reserve units have been allocated at Amersfoort (Amf), Amsterdam (Asd), The Hague (Gv), and Rotterdam (Rtd).

Finally, the fifth part of the name "P*i*" specifies the number of passengers in the system. Instances with "P1" have 422,022 passengers distributed over the day as shown in Figure 6. Instances with "P2" have 15% more passengers in all passenger groups (in total, 485,375 passengers).

8.1.2. Objective Function. The objective function used in the instances consists of the system-related

part and the service-related part. As described earlier, the service-related objective is measured by the total delay experienced by the passengers. More specifically, we minimize the sum of the delay minutes and the sum of the penalties for passengers who leave the system.

For the system-related part, we minimize with highest priority the number of cancelled trips, and with secondary priority the number of changes to the shunting processes and the number of end-of-day off-balances. More specifically, we compute the minimal number of trip cancellations in a preprocessing step, and use this as an upper bound in the main optimization models. Therefore, we do not have a specific cost parameter for the cost of the cancellation of a trip. Disruptions "D1," "D3," and "D4" do not need the cancellation of any trip, whereas disruptions "D2" and "D5" require the cancellation of one trip.

Further, we use a cost of 500 for introducing a new shunting operation or for changing the type of a shunting operation. Shunting a different number of units or cancelling a shunting operation is penalized by 100, whereas off-balances cost 400. Finally, we use a penalty of 0.0001 for the carriage kilometers to ensure that for two solutions with the same value for all other objective terms, the one with the lower operating cost is used. Note that all other objective parameters outweigh the total contribution of the carriage kilometers.

As is often the case with realistic problems, solving the MIP models to optimality is impractical. Therefore, we aborted the MIP optimization process whenever an optimality gap of 2% was proven.

For a concrete application of the approach, it is up to the decision maker to decide on the trade-off between the service- and system-related costs. However, in this study we primarily focus on the service-oriented part of the rolling stock rescheduling process, i.e., on the delays of the passengers. Therefore, we apply relatively low cost parameters to penalize the changes of system. We also carried out some experiments with different values for the system-related cost parameters, i.e., we divided or multiplied them by a factor of 10, but this resulted in very similar solutions.

Note that the trade-off between service- and system-related costs could be studied differently as well. One might minimize the passenger delay subject to an additional constraint that limits the increase of the system-related costs compared to its lowest possible value. Requiring different values for the system-related cost increase provides insight into the trade-off between the two objectives. However, as we focus on the service-oriented part of the rescheduling process, the sketched analysis lies out of our scope.

Note further that it would not be obvious to analyze the trade-off the other way around: i.e., the system-related costs are minimized and the increase of the service-related costs is limited. Indeed, the service-related costs are computed in the simulation and not in the optimization.

8.2. Results

We first compare the Boot-Opt variant and the Boot-Sim variant (see §6.4). Further, we investigate the performance of the approach for different values of the parameter α that weighs the feedback from the current iteration against feedback from earlier iterations. We then run our approach on the remaining instances using the Boot-Sim variant and with a fixed value of α.

8.2.1. Parameter α. To compare the variants Boot-Opt and Boot-Sim and to analyze the effect of the parameter α, we used both variants to optimize the instances with different values of α in the interval $0 \leq \alpha \leq 1$. In each run we performed 30 iterations, because, in some initial test experiments, this number of iterations turned out to be sufficient to find the best solution for all instances. However, for the Boot-Sim variant, this number of iterations always turned out to be unnecessarily high, so the Boot-Sim variant can be stopped earlier as well.

In Table 2 we report the performance of both variants on the five instances named "DiT1O1R2P1" where $i = 1, \ldots, 5$. The table shows the service- and system-related costs of the best solution found, as well as the iteration in which this best solution was found. We report only the results on the mentioned five instances to limit the size of the table, but it should be noted that the reported results are representative for the results on the other 25 instances for all values of α.

In particular, the best solution found by the two variants of the iterative algorithm are very similar, both for the system-related costs and for the service quality. Furthermore, for all instances and for all values of α, the Boot-Sim variant found the best solution in one of the first few iterations, after which the iterative process could not improve the solution any more. In contrast, the Boot-Opt variant often starts with an initial solution that is inferior from the service point of view and requires a significant number of improvement steps.

To a large extent, the latter is a consequence of the fact that in the Boot-Opt variant the rolling stock is rescheduled first, without considering the preferred passenger paths. This results in an initial rolling stock circulation that is away from what the passengers would wish; after that, several iterations are needed to obtain a fitting rolling stock circulation. In the Boot-Sim variant, the preferred passenger paths are generated first. In most cases, the first rescheduled rolling

Table 2 Results for the Instances "D*i*T1O1R2P1" with $i = 1, \ldots, 5$ Using Boot-Opt and Boot-Sim for Different Values of the Parameter α

		Boot-Opt			Boot-Sim		
Instance	α	Service	System	Iteration	Service	System	Iteration
D1T1O1R2P1	0.20	520,633	4,049	14	515,421	5,450	1
	0.35	520,519	4,849	14	515,421	5,450	1
	0.50	518,450	4,149	13	515,421	5,450	1
	0.65	522,452	5,849	7	515,421	5,450	1
	0.80	522,250	5,249	13	515,421	5,450	1
	1.00	536,076	5,749	4	515,421	5,450	1
D2T1O1R2P1	0.20	1,244,721	16,952	12	1,243,409	14,750	20
	0.35	1,241,796	15,751	19	1,244,753	16,151	4
	0.50	1,248,142	14,351	15	1,243,452	16,051	4
	0.65	1,241,926	16,651	7	1,244,774	15,451	4
	0.80	1,247,359	14,551	12	1,247,713	15,750	3
	1.00	1,253,500	15,651	4	1,247,158	16,250	3
D3T1O1R2P1	0.20	429,898	3,350	18	429,898	3,350	1
	0.35	429,898	3,350	10	429,898	3,350	1
	0.50	429,898	3,350	7	429,898	3,350	1
	0.65	429,898	3,350	6	429,898	3,350	1
	0.80	429,898	4,150	2	429,898	3,350	1
	1.00	429,898	4,150	2	429,898	3,350	1
D4T1O1R2P1	0.20	689,086	7,951	22	689,086	7,851	1
	0.35	689,086	10,653	4	689,086	7,851	1
	0.50	689,086	7,851	20	689,086	7,851	1
	0.65	689,086	9,953	3	689,086	7,851	1
	0.80	692,419	10,053	3	689,086	7,851	1
	1.00	721,006	5,751	6	689,086	7,851	1
D5T1O1R2P1	0.20	606,275	10,151	20	600,732	11,551	6
	0.35	606,839	10,250	15	599,888	11,851	5
	0.50	606,385	9,651	20	600,628	10,450	5
	0.65	610,253	11,051	12	600,628	10,450	5
	0.80	617,681	9,350	17	601,865	9,850	4
	1.00	699,289	12,851	6	606,488	11,451	1

Note. Each row contains, for a certain instance and value of α, the service and system objective of the best solution found and the iteration in which this solution was found.

stock circulation can already facilitate the passenger demand as well as our multi-iteration framework can.

The different choices of α can mainly be distinguished by the results obtained with Boot-Opt. We find that the results for $\alpha = 1.00$ are consistently worse than the results for all other values of α. This is not surprising, because only the feedback from the last iteration is taken into account in each step, and we therefore do not use the information obtained in the earlier iterations. In fact, for the instances "D1"–"D4" we observe cyclical behavior of the algorithm. After a few iterations, the algorithm alternates between the same two solutions.

For the instance "D5T1O1R2P1" we plotted the traversal of the results of the Boot-Opt variant in the two-dimensional objective space in separate diagrams in Figure 7 for each value of the parameter α. Figures representing similar results for other instances can be found in Nielsen (2011).

In Figure 7 we observe that for smaller values of α the feedback mechanism is more effective than for larger values. For the instance "D5T1O1R2P1," values of α up to 0.50 lead to a certain kind of convergence to a rolling stock circulation with low service-related costs and low system-related costs. This convergence is strongest for $\alpha = 0.20$ and $\alpha = 0.35$. However, for other instances we observed that for $\alpha = 0.20$ the iterative procedure takes smaller steps in the objective space, so that it takes longer to reach the best solution. We therefore use the value $\alpha = 0.35$ for the remaining experiments. Also, for the Boot-Sim variant this turned out to be an appropriate value for α.

8.2.2. All Instances. The results of our further computational experiments on all instances for $\alpha = 0.35$ with the Boot-Sim variant are shown in Table 3. The final results obtained with the Boot-Opt variant were very similar to the results obtained with the Boot-Sim variant, but they required significantly more computation time.

Table 3 summarizes the characteristics of two solutions for each of the test instances: BestSol is the solution with the lowest total cost value (after

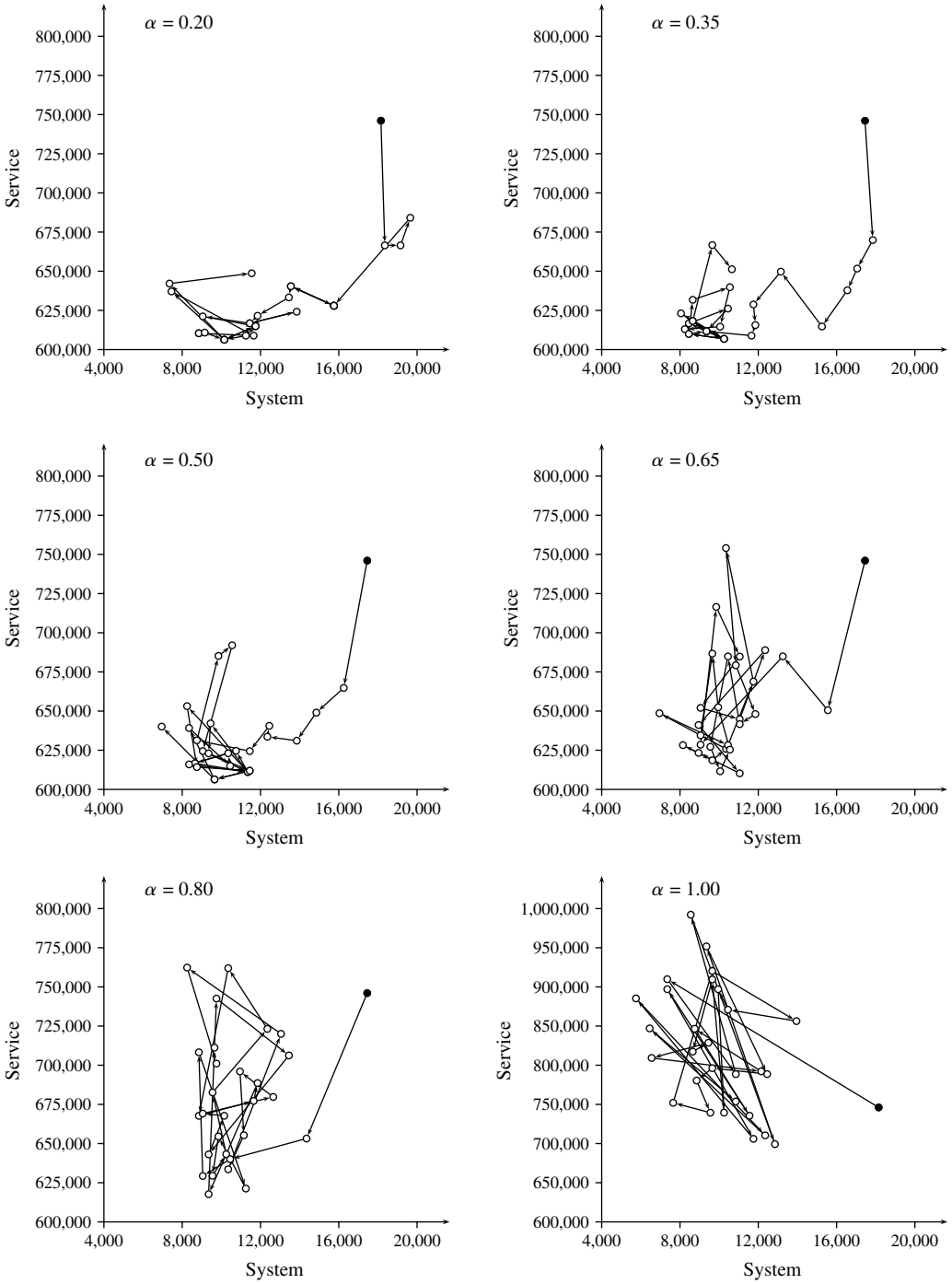

Figure 7 Traversal of the Objective Space for Different Settings of Parameter α for Instance "D5T101R2P1" Using Boot-Opt

Notes. The first iteration is marked by a solid dot. Note that the scale of the y-axis differs for $\alpha = 1.00$.

30 iterations), while SystemSol is obtained by minimizing the system-related objective terms only.

The first column in Table 3 contains the instance names. The next three columns under "OPT time" show the minimum, average, and maximum experienced computation times (in seconds) for the optimization module over 30 iterations. The next column under "SIM" shows the average computation time (in seconds) for the simulation module of the algorithm. The next two columns under "Time" indicate the total computation time (in seconds) for all 30 iterations and for reaching BestSol, respectively, including the time for the passenger simulation. Recall that we use an optimality gap of 2% in each optimization run.

The columns "Service" and "System" under "SystemSol" and "BestSol" give the service and system

Table 3 Results for All Instances with Boot-Sim

Instance	OPT time Min	OPT time Avg	OPT time Max	SIM time	Time Total	Time Best	SystemSol Service	SystemSol System	BestSol Service	BestSol System	Lower bound IC-LB	Lower bound WOC-LB	Service impr. (%)
D1T1O1R1P1	1.1	1.6	11.3	0.4	58.7	11.7	628,211	1,449	582,215	5,950	421,007	544,674	7
D1T1O1R2P1	1.1	1.5	7.9	0.3	56.1	8.3	578,550	1,249	515,421	5,450	421,007	430,681	11
D1T1O1R3P1	1.0	1.5	7.6	0.3	54.6	7.9	579,315	1,249	473,786	5,850	421,007	424,816	18
D1T1O2R1P1	1.0	1.4	6.9	0.4	53.9	7.2	657,197	2,049	618,346	5,650	421,007	556,666	6
D1T1O1R2P2	1.0	1.5	7.7	0.4	55.8	8.1	818,667	1,249	728,725	6,750	484,126	532,481	11
D1T2O1R1P1	1.7	2.4	15.8	0.4	82.1	16.3	303,037	1,149	287,612	3,049	287,612	287,612	5
D2T1O1R1P1	3.3	3.8	11.7	0.4	126.5	20.2	1,818,607	4,248	1,247,016	16,851	996,484	1,039,719	31
D2T1O1R2P1	3.3	3.7	10.0	0.4	123.7	21.8	1,818,607	4,248	1,244,753	16,151	996,484	1,039,719	31
D2T1O1R3P1	3.1	3.5	8.6	0.4	116.7	16.4	1,818,607	4,248	1,242,230	15,151	996,484	1,039,719	31
D2T1O2R1P1	3.3	3.7	8.8	0.4	122.5	40.5	1,805,419	3,849	1,302,372	19,651	996,484	1,068,677	28
D2T1O1R2P2	3.1	3.6	10.3	0.4	119.1	10.7	2,567,406	4,248	1,631,860	17,350	1,145,355	1,263,904	36
D2T2O1R1P1	10.6	11.2	22.9	0.4	345.4	34.4	1,115,628	3,248	744,419	8,650	721,884	727,088	33
D3T1O1R1P1	1.1	1.5	6.3	0.3	55.0	6.7	505,971	2,349	429,898	3,750	416,595	426,838	15
D3T1O1R2P1	1.1	1.5	7.1	0.3	56.0	7.5	505,971	2,049	429,898	3,350	416,595	426,838	15
D3T1O1R3P1	1.1	1.5	8.1	0.3	56.5	8.5	505,971	2,049	429,898	3,350	416,595	426,838	15
D3T1O2R1P1	1.1	1.5	7.4	0.3	55.8	7.7	527,005	3,049	429,898	5,350	416,595	426,838	18
D3T1O1R2P2	1.1	1.6	7.2	0.3	58.7	12.2	765,566	2,049	514,276	6,851	479,347	500,599	33
D3T2O1R1P1	1.7	2.4	14.0	0.4	84.4	24.7	625,948	949	454,710	4,450	444,589	450,480	27
D4T1O1R1P1	1.0	1.4	6.5	0.3	52.7	8.1	839,882	1,349	698,616	7,451	666,626	674,203	17
D4T1O1R2P1	1.0	1.4	6.2	0.3	53.3	6.6	839,882	1,349	689,086	7,851	666,626	670,784	18
D4T1O1R3P1	1.0	1.5	7.8	0.3	54.5	8.1	839,882	1,349	686,646	8,451	666,626	670,784	18
D4T1O2R1P1	1.0	1.5	7.0	0.3	55.7	7.4	842,661	1,249	702,098	9,451	666,626	675,355	17
D4T1O1R2P2	1.1	1.5	6.4	0.4	55.7	6.8	1,191,093	1,349	841,656	12,052	766,343	785,204	29
D4T2O1R1P1	1.6	2.4	11.9	0.4	81.8	12.2	494,192	1,048	459,829	6,151	459,536	459,572	7
D5T1O1R1P1	4.4	5.1	10.5	0.3	164.4	81.6	1,116,650	1,449	606,120	11,151	520,867	536,397	46
D5T1O1R2P1	4.1	4.6	10.3	0.4	148.6	28.6	1,116,650	1,449	599,888	11,851	520,867	536,397	46
D5T1O1R3P1	4.3	4.9	9.8	0.3	157.3	10.1	1,116,650	1,449	593,361	12,551	520,867	536,397	47
D5T1O2R1P1	4.4	5.1	16.3	0.4	163.4	71.9	1,122,055	1,449	671,919	15,551	520,867	536,397	40
D5T1O1R2P2	4.2	4.5	10.1	0.4	146.8	33.6	1,722,815	1,449	922,617	16,852	599,038	635,239	46
D5T2O1R1P1	12.9	13.7	25.6	0.4	422.7	25.9	754,466	549	410,374	9,151	388,216	388,216	46

Notes. The table shows computation times for the optimization module, for the simulation module, for all 30 iterations, and for reaching BestSol. It also contains the objective values of SystemSol and BestSol, as well as the lower bounds based on infinite capacity (IC-LB) and on weak operator control (WOC-LB). The last column indicates the relative difference between the service objectives of BestSol and SystemSol.

objectives of SystemSol and BestSol, respectively. The columns "IC-LB" and "WOC-LB" under "Lower bound" indicate the two types of lower bounds on the service objective that were described in §7. Finally, column "Service impr." shows the relative improvement of the service objective when moving from SystemSol to BestSol.

We can make a number of observations concerning the results. First, we observe that our approach is able to improve the service quality significantly in all instances at the cost of a limited number of changes to the system. The service improvements are between 5% and 47%, as shown in the last column of Table 3. From a practical point of view, the number of changes to the system are appealing, especially considering the fact that the changes are divided among 14 stations over more than eight hours. Indeed, for disruptions "D1" and "D3" we have 2–7 new shunting operations, 1–10 slightly changed ones, and 8–14 cancelled shunting operations. The solutions have 1–4 off-balances. For disruptions "D2," "D4," and "D5," we generally experience more changes to the system; 8–22 new shunting operations, 4–33 slightly changed ones, and 8–28 cancelled ones. The solutions for these disruptions have 0–11 off-balances. Note that these detailed figures are not shown in Table 3.

Second, for disruptions "D3" and "D4" we see that the service objective of the best solutions found is quite close to the lower bound given by the WOC-LB. This means that most of the experienced delay is due to the modified timetable rather than lack of capacity. For all other instances we experience that the modifications to the timetable are still by far the major contributors to the passenger delay. Lack of capacity contributes to 6–31% of the passenger delay. This is in line with the second observation mentioned above.

Third, based on the results for the instances "R1" with no reserve units, "R2" with three reserve units, and "R3" with six reserve units, we note that we are able to alleviate more passenger delay in the instances with more available reserve units. The results for these instances are the first, second, and third rows,

respectively, in each block in Table 3. In particular, disruption "D1" (between The Hague (Gv) and Rotterdam (Rtd)) appears to benefit from the extra reserve units.

Fourth, we notice that there is a significant difference between disruptions in the peak hours and in the off-peak hours. This is apparent when comparing the disruptions at time 16:00–19:00 (named "T1") to the disruptions at time 11:00–15:00 (named "T2")—the first and sixth row in each block in Table 3, respectively. Generally, fewer delay minutes occur in a four-hour off-peak disruption compared to a three-hour disruption during the peak. Also, it is possible for all disruptions "D1,"...,"D5" to bring the service objective close to the IC-LB in the off-peak instances. This indicates that capacity is not a bottleneck in those situations.

Fifth, we observe that the instances with less slack capacity in the original plan generally lead to rescheduled solutions with worse service objectives. This concerns "O2" (fourth row in each block in the table), which was planned with less rolling stock than "O1" (first row), and "P2" (fifth row), which has 15% more passengers than "P1" (second row) but uses the same amount of rolling stock.

Sixth, WOC-LB spectacularly improves on IC-LB in two cases (D1T1O1R1P1 and D1T1O2R1P1); the improvement in other cases is limited.

8.2.3. Graphical Representation of Passenger Delay. Figure 8 plots, for five of the tested disruptions, the number of delayed passengers per time instant. These values are computed as follows. Suppose that no disruption takes place on the considered day; then, every passenger is assumed to travel on the shortest path in the undisrupted timetable. In case of a disruption, the passengers are assumed to enter the system at the same time as they would without a disruption. Furthermore, the travel options in the disrupted timetable are no shorter than those in the undisrupted timetable.

Therefore, for any time instant, the number of passengers in the system in *any* realization of the passenger flows on a day with a disruption exceeds the number of passengers in the system on the corresponding undisrupted day (even if infinite capacities of the trains are assumed). The difference between the two numbers of passengers indicates the number of passengers in the system at that time instant that are delayed by the disruption.

The number of delayed passengers in the system on the undisrupted day is 0, which is represented by the horizontal time axis. Furthermore, each diagram in Figure 8 contains a curve for the number of delayed passengers subject to three capacity assignments for the rolling stock: (i) infinite capacities of the trains (white; this corresponds to IC-LB), (ii) the best found solution (dark gray), and (iii) the solution that is best for the system (light gray).

The curve for IC-LB indicates the passenger delays that are caused by the cancellations of trains in the disrupted timetable; no rolling stock assignment can improve on it. Actually, at any time instant the curve for IC-LB lies under the curve for any other solution.

We notice that for the complicated disruptions "D2" and "D5" there is a significant gap between the best found solution and the solution that is best for the system. For the disruptions "D3" and "D4," the best solution is almost identical to the lower bound. Finally, we observe that for disruption "D1," the best solution contains more delayed passengers than the best system solution at some time instants. However, when the disruption is over, the best solution brings the delayed passengers out of the system faster than the solution that is best for the system.

8.2.4. Computation Times. The computation times of the optimization module and the simulation module for each instance are shown in Table 3, considering a 2% optimality gap in each optimization run. The computation times of the optimization module seem to be instance dependent. For instances involving disruptions in the evening peak ("T1"), we experienced computation times of up to 23 seconds for one iteration, but usually less than 15 seconds, the average being 1.2–5.1 seconds. Instances involving the early disruption "T2" need somewhat more computation time, because the rolling stock must be rescheduled during a larger part of the day.

Recall that the Boot-Sim variant is already able to find a very good solution in the first few iterations. In addition, the current implementations of both variants based on CPLEX 12.4 have promisingly low average and maximum computation times per iteration. As a consequence, the time required to find the best solution with the Boot-Sim variant is always less than 90 seconds. We conclude that the Boot-Sim variant is an appropriate tool for the time-pressured environment of real-life disruption management.

Finally, we want to mention that the computation times did have a few extreme outliers when using the earlier CPLEX 11 for solving the MIP model. Although the typical computation times were comparable with the values reported earlier, one particular optimization step took 3,300 seconds, with most of the time actually being used to find the first reasonable solution. However, the use of CPLEX 12.4 eliminated this problem for our test instances completely.

We note that, if that might be necessary, then extremely long computation times can also be handled by using a rolling horizon approach rather than computing the rolling stock circulation in one attempt, as is described by Nielsen, Kroon, and Maróti (2012). This is a subject for further research.

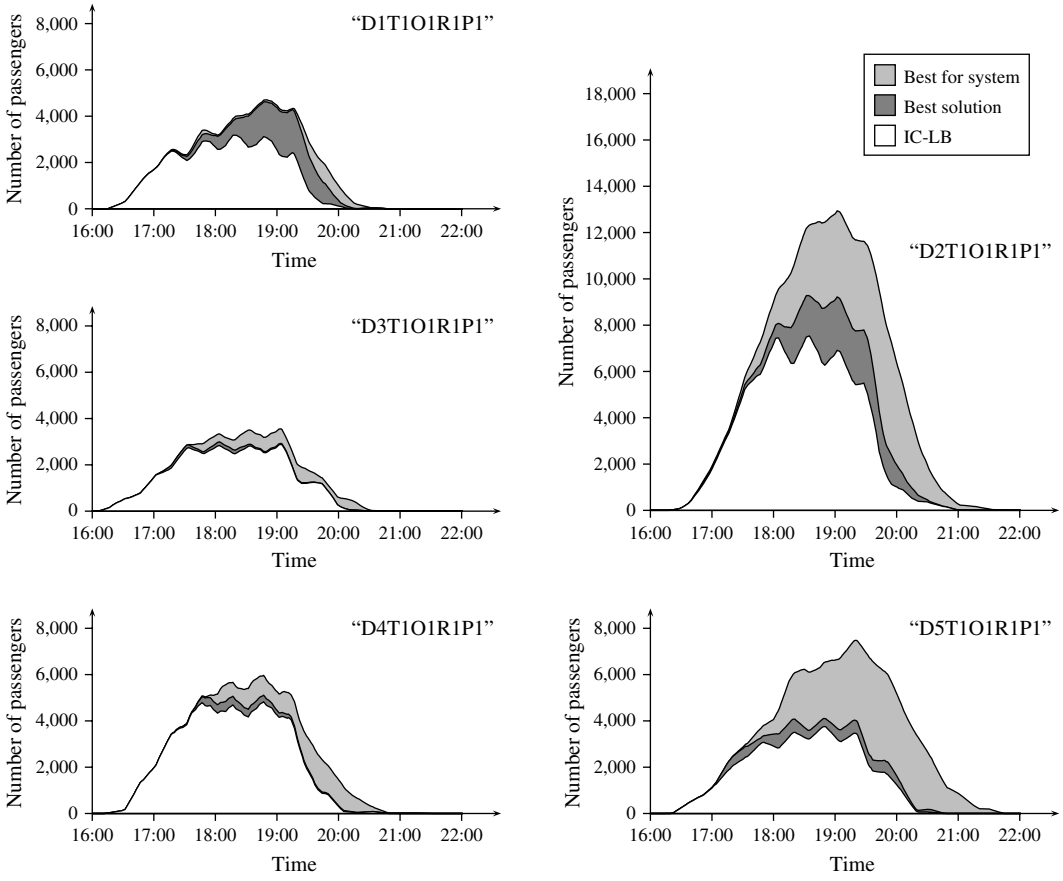

Figure 8 The Numbers of Delayed Passengers in the System During and After the Disruption for the Instances "D1T1O1R1P1," ..., "D5T1O1R1P1"
Note. The area under a curve represents the cumulative delay.

9. Conclusions and Future Research

In this paper we describe an iterative approach for improving the service for the passengers during major disruptions of the railway system by adapting the rolling stock circulation to the *modified* passenger flows. The passenger flows are modified because, due to the disruption, several trains have been cancelled or rerouted, and passengers use alternative routes to their destinations.

For all instances in the computational experiments, we are able to improve the service objective, and in some cases we can even reach the described lower bound. The improvements in service for the passengers require some additional changes to the rolling stock circulation.

Our computational results show that the additional service for the passengers resulting from our approach is especially noticeable during the peak hours, when seat capacity is scarce. During the off-peak hours, many trains still have some slack capacity, and therefore the modified passenger flows can be accommodated more easily without rescheduling the rolling stock too much. However, because of the increased efficiency targets of NS, the slack capacity of *all* trains will be reduced in the near future. The application of our approach will then be even more effective than it is in the current situation. Therefore, our models will be implemented into user-friendly decision support tools to be used in the real-time operations.

In this paper we describe two variants for starting the iterative algorithm: the Boot-Sim variant starts with a simulation of the passenger flows, and the Boot-Opt variant starts with an optimization of the rolling stock circulation. It turned out that Boot-Sim usually outperforms Boot-Opt, because Boot-Sim provides a very good solution in one of the first iterations already, whereas Boot-Opt needs a higher number of iterations to find a solution of comparable quality. Especially for the Boot-Sim variant, the computation times are appealing for real-time use.

If needed, a possible way to reduce the computation time would be to utilize the rolling horizon framework introduced by Nielsen, Kroon, and Maróti (2012). This framework would also allow accounting for the uncertainty of the system during a disruption, for example, related to the uncertain duration of the disruption. However, we have chosen not to utilize this rolling horizon framework for this rescheduling study with dynamic passenger flows, because

it would obscure the contributions of the iterative framework described in this paper.

In our future research, we will investigate approaches to compute the strong operator controlled lower bound (SOC-LB). It is appealing to develop a cut-and-price approach for solving the underlying model. Such an approach would combine column generation for the paths of the passengers, and valid cut generation based on Benders decomposition for the assignment of rolling stock capacity.

References

Ahuja R, Magnanti T, Orlin J (1993) *Network Flows: Theory, Algorithms and Applications* (Prentice-Hall, Englewood Cliffs, NJ).

Ball M, Barnhart C, Nemhauser G, Odoni A (2007) *Air Transportation: Irregular Operations and Control, Handbooks in Operations Research and Management Science*, Vol. 14 (North-Holland, Amsterdam).

Bratu S, Barnhart C (2006) Flight operations recovery: New approaches considering passenger recovery. *J. Scheduling* 9(3): 279–298.

Budai G, Maróti G, Dekker R, Huisman D, Kroon LG (2009) Rescheduling in passenger railways: The rolling stock rebalancing problem. *J. Scheduling* 13(3):1–17.

Cacchiani V, Caprara A, Galli L, Kroon L, Maróti G, Toth P (2012) Railway rolling stock planning: Robustness against large disruptions. *Transportation Sci.* 46(2):217–232.

Clausen J, Larsen A, Larsen J, Rezanova NJ (2010) Disruption management in the airline industry—Concepts, models and methods. *Comput. Oper. Res.* 37(5):809–821.

Dumas J, Soumis F (2008) Passenger flow model for airline networks. *Transportation Sci.* 42(2):197–207.

Dumas J, Aithnard F, Soumis F (2009) Improving the objective function of the fleet assignment problem. *Transportation Res. Part B* 43(4):466–475.

Fioole PJ, Kroon LG, Maróti G, Schrijver A (2006) A rolling stock circulation model for combining and splitting of passenger trains. *Eur. J. Oper. Res.* 174(2):1281–1297.

Jespersen-Groth J, Clausen J (2006) Optimal reinsertion of cancelled train lines. Technical report, Informatics and Mathematical Modelling, Technical University of Denmark, Lyngby, Denmark.

Jespersen-Groth J, Potthoff D, Clausen J, Huisman D, Kroon LG, Maróti G, Nielsen MN (2010) Disruption management in passenger railway transportation. Ahuja R, Möhring R, Zaroliagis C, eds. *Special Volume on Robust and Online Large-Scale Optimization* (Springer, Berlin), 399–422.

Kohl N, Larsen A, Larsen J, Ross A, Tiourine S (2007) Airline disruption management—Perspectives, experiences and outlook. *J. Air Transport Management* 13(3):149–162.

Kroon L, Huisman D, Abbink E, Fioole P-J, Fischetti M, Maróti G, Schrijver A, Steenbeek A, Ybema R (2009) The new Dutch timetable: The OR revolution. *Interfaces* 39(1):6–17.

Nielsen LK (2011) Rolling stock rescheduling in passenger railways. Ph.D. thesis, Erasmus University, Rotterdam, The Netherlands.

Nielsen LK, Kroon LG, Maróti G (2012) A rolling horizon approach for disruption management of railway rolling stock. *Eur. J. Oper. Res.* 220(2):496–509.

Potthoff D, Huisman D, Desaulniers G (2010) Column generation with dynamic duty selection for railway crew rescheduling. *Transportation Sci.* 44(4):493–505.

Yan S, Tang C, Shieh C-L (2005) A simulation framework for evaluating airline temporary schedule adjustments following incidents. *Transportation Planning Tech.* 28(3):189–211.

Yu G, Qi X (2004) *Disruption Management: Framework, Models and Applications* (World Scientific, Singapore).